

Documentation project Billard 2025 BA3

Authors: Arun Gemsch, Leon Kesslering, Adarsh Ravikumar, Patrick Stadler

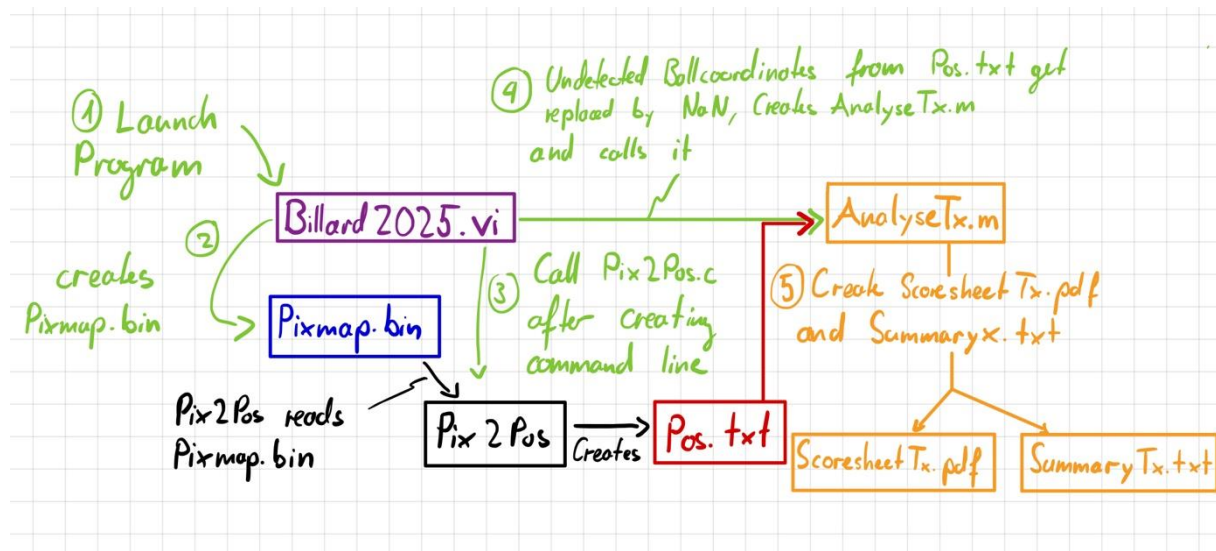
Platform: macOS Version: 15.6.1 (24G90)

C Editor + Compiler: Visual Studio Code Version: 1.107.0 ; clang version 17.0.0

Matlab + Version: MATLAB_R2025b

LabView + Version: NI Labview Community edition 2025 Q3 (64-bit)

Data flow between applications



List of LabView Files and description of SubVIs

MyFourthVI: Vi takes RGB Value Inputs and image from the sequence to calculate Xmin, Xmax, Ymin, Ymax for each picture. We loop over every pixel of the picture and compare with the RGB values for the board and extract the values.

BilliardVI: Main Vi where C input is created and C consequently executed. From then on it's results are extracted and further used to call matlab and create summary and open the pdf if is wished for

Matlab String (Sub)VI: We extract the RGB values for the band touch lose/win and create the matlab string which consists of the extracted RGB values set by the operator and our main matlab code.

List of C files:

Pix2Pos.c: The C code that takes Pixmap.bin as input, searches it for the three balls, and returns the Pos.txt file, which contains the coordinates and scores of the three balls.

Pixmap.bin: Is a binary file that contains the image width, height, and all pixel values in the 0x00RRGGBB format.

Pos.txt: Text file that contains X and Y coordinates and scores of the balls.

Description of the C algorithm :

□ First, the command-line arguments are validated (the program expects exactly 29 parameters) and then converted to numeric values and stored in the corresponding variables (billiard box bounds, RGB ranges for each ball color, background RGB range, and the ball diameter). The ball diameter is checked to be within the allowed range (10–15 pixels).

□ Next, the file Pixmap.bin is opened in binary mode. The program reads the image width and height from the first two unsigned integers, validates them, and then reads the pixel data into a dynamically allocated 2D matrix (height × width). The program also checks whether the file contains too few pixels (error) or extra data after the expected image size (warning).

□ The program then searches for the most likely positions of the red, yellow, and white ball within the specified “billiard box” region. Two nested loops move a square window of size BallDiameter × BallDiameter across the image (the stored coordinates correspond to the top-left corner of that window).

To speed up the search, the program first checks the pixel at the center of the window using isBackground(). If the center pixel matches the background color range, the window is skipped immediately.

Otherwise, testSquare() computes a score for each ball color by counting how many pixels inside the window fall within that ball’s RGB range. If a score is higher than the best score found so far for that color, the program updates the best score and stores the corresponding coordinates.

□ After scanning the entire region, the program verifies whether each best score reaches the minimum detection threshold BallminScore. If a ball’s score is below the threshold, that ball is considered “not detected” and its coordinates are set to (-1, -1) and its score to 0.

□ Finally, the detected positions and scores for the three balls are written to Pos.txt.

Throughout the program, multiple error checks are performed (invalid parameters, invalid image dimensions, allocation failures, file read errors, etc.).

Error handling in C:

Info: In case of an error, the program ensures that all open files are closed and all dynamically allocated memory is released before exiting.

□ Incorrect number of command-line arguments → **ERROR**: program stops with an error code.

□ Ball diameter outside the allowed range (10–15 pixels) → **ERROR**: program stops with an error code.

□ Failure to open the input image file Pixmap.bin → **ERROR**: program stops with an error code.

□ Failure to read image width or height from the file → **ERROR**: program stops with an error code.

- ❑ Image width or height outside allowed limits → **ERROR**: program stops with an error code.
- ❑ Invalid billiard box parameters (negative values or min > max) → **ERROR**: program stops with an error code.
- ❑ Billiard box exceeds the image boundaries → **ERROR**: program stops with an error code.
- ❑ Failure to allocate memory for the image matrix → **ERROR**: program stops with an error code after freeing allocated memory.
- ❑ Fewer pixels read than expected from the image file → **ERROR**: program stops with an error code and frees allocated memory.
- ❑ Additional data detected after the expected number of pixels → **WARNING**: extra data is ignored.
- ❑ Red ball score below the minimum detection threshold → **WARNING**: red ball is marked as not detected.
- ❑ Yellow ball score below the minimum detection threshold → **WARNING**: yellow ball is marked as not detected.
- ❑ White ball score below the minimum detection threshold → **WARNING**: white ball is marked as not detected.
- ❑ Failure to open the output file Pos.txt for writing → **ERROR**: program frees the allocated memory and then stops with an error code.

Description of Matlab file and it's functions:

1. Main analysis script (AnalyseTx.m)

Purpose

This is the main MATLAB script that analyses a billiard sequence and produces the final results (figure, PDF, and summary text file).

What it does

- Loads the ball trajectories (Red, Yellow, White) from TracesT1.m.
- Cleans the data by:
 - Replacing missing values (NaN).
 - Removing outliers.
- Converts the coordinate system from LabVIEW to MATLAB.
- Computes the billiard frame that contains all ball trajectories.
- Computes:
 - Distance travelled by each ball.
 - Order in which the balls start moving.
 - Contacts between the first ball and the cushions.

- Determines whether the player **wins or loses** according to the rules.
- Displays the result graphically.
- Exports:
 - **ScoreSheetNomDeLaSequence.pdf** (Graphical summary of the game).
 - **SummaryNomDeLaSequence.txt** (Compact machine-readable summary)

2. InterpolateNan(X, Y)

Purpose

Replaces missing coordinates (NaN) by the *next valid value* in time.

What it does

- Detects NaN values in X and Y.
- Uses linear interpolation with the 'next' method.
- Ensures the first and last samples are valid by assigning default values if needed.

3. RemoveOutlier(X, Y)

Purpose

Removes isolated erroneous measurements (outliers).

What it does

- Detects outliers independently in X and Y using a moving median.
- Marks a point as an outlier only if **both X and Y are outliers**.
- Replaces the outlier value with the previous valid coordinate.

4. GetFrame(Xr, Yr, Xy, Yy, Xw, Yw)

Purpose

Computes the smallest rectangular frame that contains all ball trajectories.

What it does

- Finds the global minimum and maximum of all X and Y coordinates.
- Returns Xmin, Xmax, Ymin, Ymax.

5. GetBallPathLength(X, Y)

Purpose

Computes the total distance travelled by one ball.

What it does

- Computes the distance between consecutive points.
- Sums all segment lengths.
- Outputs the total path length in pixels.

6. GetFirstMoveIdx(X, Y, MoveDistPx)

Purpose

Finds when a ball starts moving.

What it does

- Computes the distance from the initial position for every time step.
- Returns the first index where the distance exceeds MoveDistPx.
- Also returns the vector of segment distances.
- If the ball never moves, an empty index [] is returned.

7. GetTouchIdx(X, Y, Xmin, Xmax, Ymin, Ymax, BallBorderDist)

Purpose

Detects contacts between a ball and the billiard cushions.

What it does

- Checks proximity to each of the four borders.
- Detects the **first index** of each contact sequence.
- Merges and sorts all detected contacts.
- Removes index 1 if present.
- Outputs a vector of time indices where cushion contacts occur.

8. GetBallMoveOrder(Xr, Yr, Xy, Yy, Xw, Yw, MoveDistPx)

Purpose

Determines which ball moves first, second, and last.

What it does

- Uses GetFirstMoveldx for each ball.
- Sorts balls by:
 - Earliest movement index.
 - Largest initial movement if tied.
- Assigns a very large index to balls that never move.
- Outputs : FirstBall, SecondBall, LastBall (1 = Red, 2 = Yellow, 3 = White) and NbBallsMoved(Number of Balls moved).

9. GetWinnerScore(idxFirstR, idxFirstY, idxFirstW, IdxTouchfirst, NbBallsMove)

Purpose

Determines whether the player wins or loses.

Rule implemented

- All three balls must move.
- The first ball must touch **at least three cushions** between the second and third ball movements.
- Returns 'win' or 'lose'.